

Particionamento de dados e balanceamento de carga

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade.

Desconsidere a pressão e quaisquer forças externas. Utilize diferenças finitas para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das cláusulas `schedule` e `collapse` no desempenho da execução paralela.

2 Fundamentação teórica

Considerando apenas a viscosidade, usa-se $\partial u / \partial t = \nu \nabla^2 u$. Euler explícito e diferenças centrais formam o stencil de cinco pontos:

$$u_{i,j}^{n+1} = u_{i,j}^n + r(u_{i-1,j}^n + u_{i+1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n), \quad r = \nu \Delta t / h^2.$$

Com $\nu = 0,1$, $\Delta t = 0,2$ e $h = 1$, $r = 0,02$, dentro do limite de estabilidade bidimensional. A gaussiana se espalha e reduz seu pico; campos nulo e constante permanecem invariantes.

As cláusulas OpenMP definem como as iterações são distribuídas. Cada política tem um custo de coordenação e uma unidade de trabalho próprios.

2.1 `schedule(static)`

A divisão das iterações é feita antes do início do laço. Cada thread recebe uma parte previsível do trabalho e não precisa solicitar novos chunks durante a execução. Essa política é adequada quando as células têm custo semelhante, como no stencil desta tarefa.

2.2 `schedule(dynamic,k)`

As threads recebem novos chunks de até k iterações quando terminam o anterior. A distribuição pode compensar diferenças de carga, mas cada solicitação acrescenta coordenação. O valor de k conta iterações do laço distribuído.

2.3 `schedule(guided,k)`

O escalonador começa entregando chunks maiores e reduz seus tamanhos à medida que o laço avança, sem ficar abaixo de k iterações. Essa combinação reduz solicitações no início e conserva alguma redistribuição no final.

2.4 `collapse(n)`

Os n laços aninhados mais externos passam a formar um único espaço de iterações. Sem a cláusula, a distribuição considera as iterações do laço externo; com ela, cada iteração lógica combina os índices dos n laços. Portanto, a mesma configuração de chunk pode representar volumes de cálculo diferentes e deve ser comparada junto com a política de `schedule`.

A Figura 1 compara as políticas de distribuição e mostra como `collapse(2)` altera a unidade do chunk.

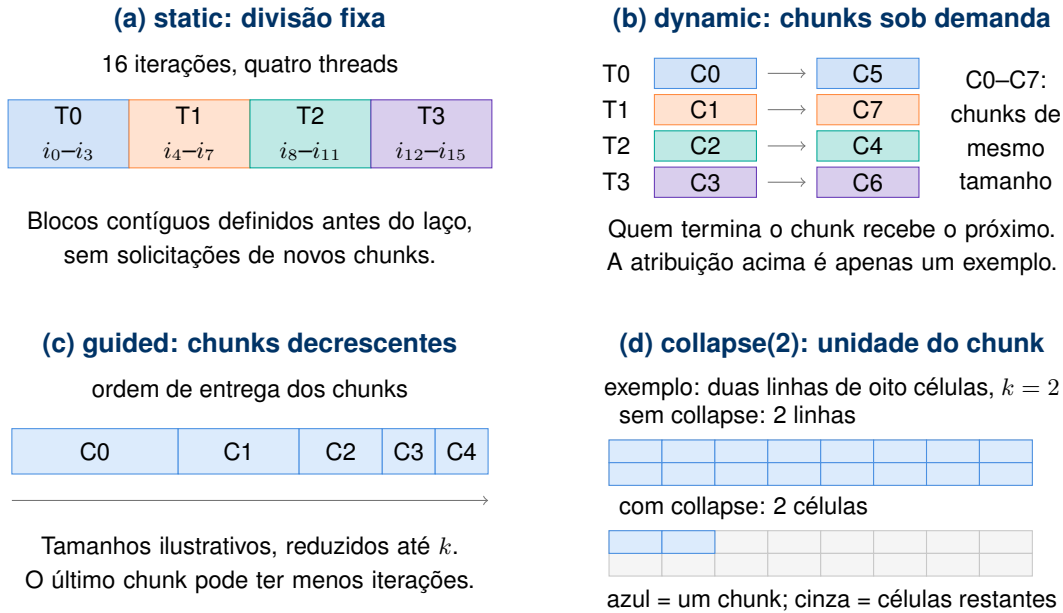


Figure 1: Distribuição ilustrativa do trabalho. T identifica threads e C identifica chunks; os quadros não representam tempos de execução.

3 Experimento

Foram usadas uma malha 512×512 , 500 passos, GCC 13.3.0, `-O2 -Wall -Wextra -lm -fopenmp` nas versões paralelas, no Intel Core i7-14700HX sob WSL2. `omp_get_wtime()` mede somente stencil, barreiras e troca de ponteiros. Inicialização, diagnósticos e exportação ficam fora. Cada ponto é a mediana de cinco repetições, com intervalo de um segundo entre medições.

Estratégia	Política	Unidade distribuída
Sequencial	sequencial	—
Static	static	linha
Static com collapse	static, collapse(2)	célula
Dynamic	dynamic, 16	16 linhas
Dynamic com collapse	dynamic, 16, collapse(2)	16 células
Guided com collapse	guided, 16, collapse(2)	chunks decrescentes de células
Guided	guided, 16	chunks decrescentes de linhas

Table 1: Variantes comparadas.

4 Implementação

Cada passo lê somente u e escreve em v . A troca dos dois ponteiros após o passo evita misturar instantes. As células do mesmo passo têm escritores distintos, mas o passo seguinte depende do anterior (RAW); por isso, apenas os laços espaciais são distribuídos.

```

1 #pragma omp parallel
2 for (t = 0; t < num_passos_tempo; t++) {
3     #pragma omp for schedule(...)
4     for (i = 1; i < N-1; i++) for (j = 1; j < N-1; j++) /* stencil */
5     #pragma omp single
6     { tmp = u; u = v; v = tmp; }
7 }

```

Listing 1: Estrutura comum às versões paralelas.

As barreiras implícitas de `omp for` e `single` garantem que a saída esteja completa antes da troca e que a troca termine antes do próximo passo.

5 Resultados

Os campos nulo e constante permaneceram inalterados. As versões paralelas reproduziram o campo gaussiano da referência sequencial com 1, 2 e 4 threads.

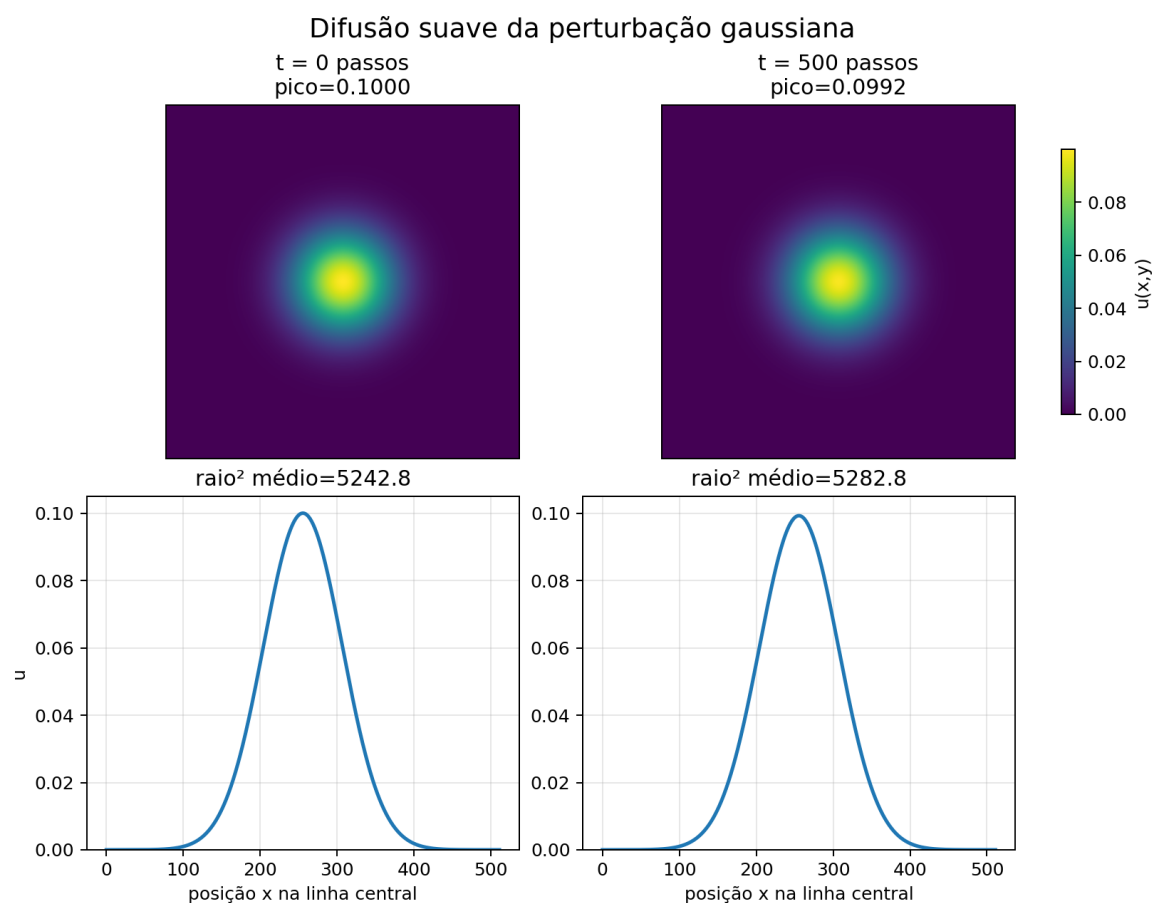


Figure 2: Difusão da perturbação gaussiana ao longo dos 500 passos. O pico diminui e se espalha, enquanto as bordas permanecem fixas.

Estratégia	1	2	4	8	14
Sequencial	0,0818	—	—	—	—
Static	0,0726	0,0420	0,0263	0,0155	0,0159
Static com collapse	0,1554	0,0881	0,0573	0,0327	0,0299
Dynamic	0,1087	0,0740	0,0444	0,0244	0,0219
Dynamic com collapse	0,2693	0,4002	0,3510	0,3252	0,3364
Guided com collapse	0,1207	0,0784	0,0429	0,0172	0,0166
Guided	0,0696	0,0403	0,0222	0,0141	0,0159

Table 2: Tempo mediano em segundos; os números das colunas são threads.

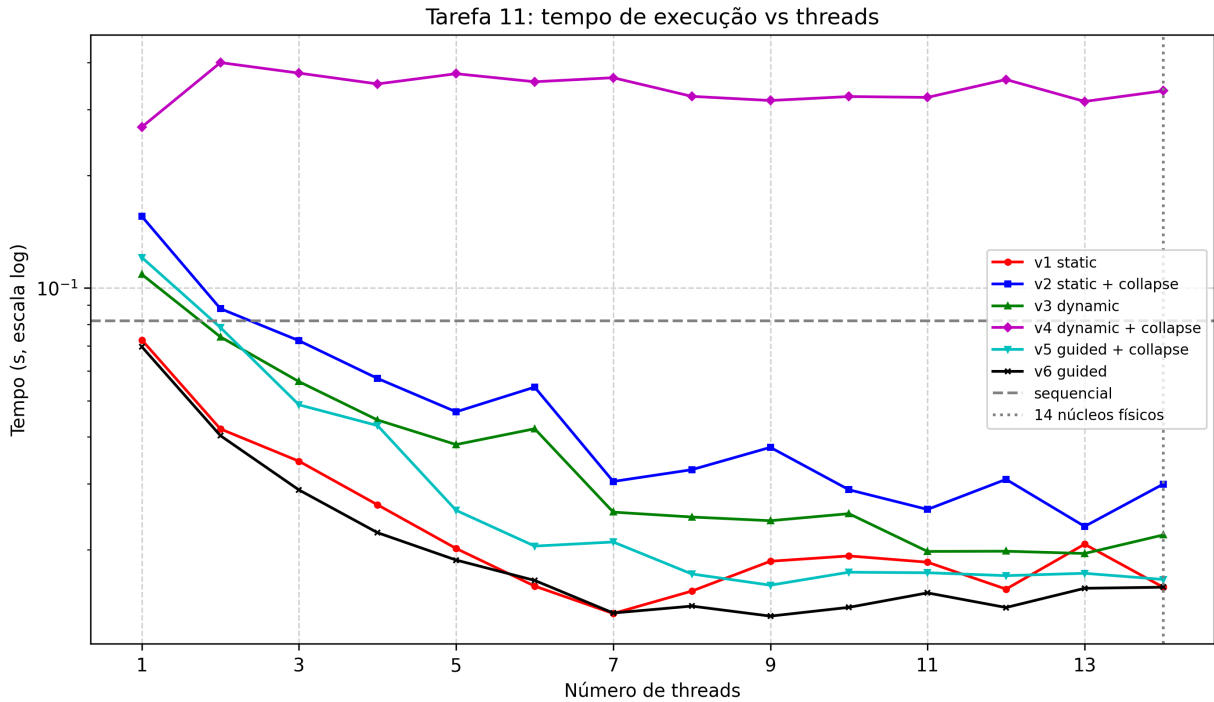


Figure 3: Tempo mediano em escala logarítmica.

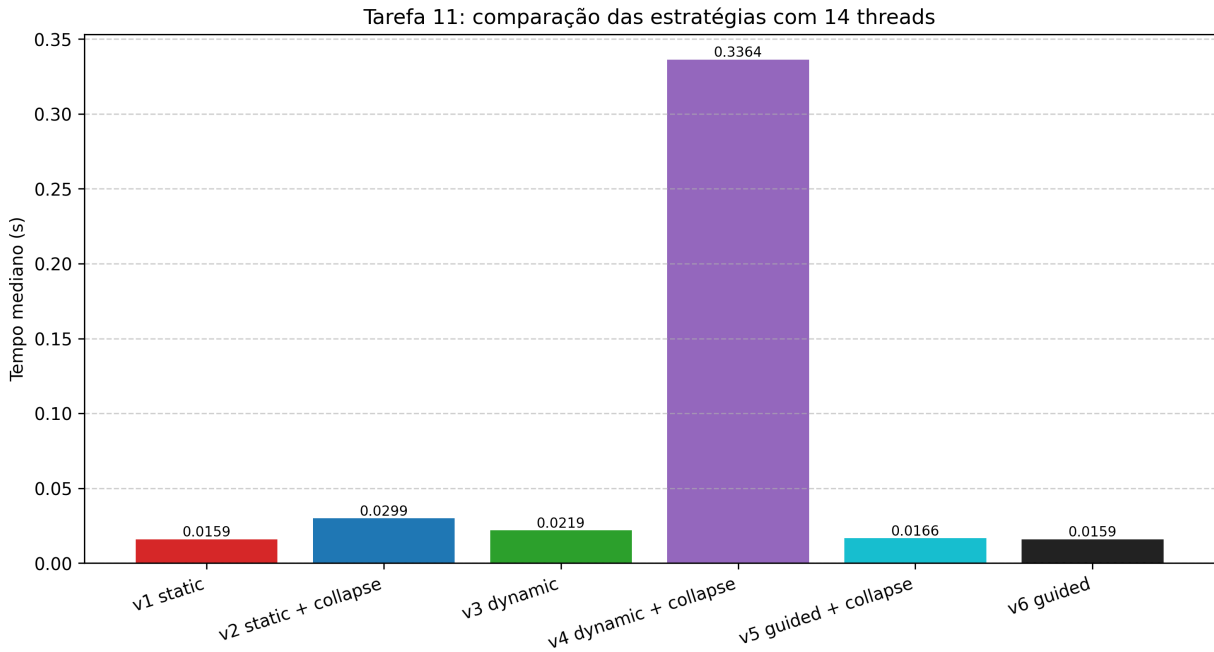


Figure 4: Comparação das estratégias com 14 threads, correspondente ao número de núcleos físicos do ambiente.

Desempenho e distribuição do trabalho. A estratégia *guided* foi a mais rápida na varredura até 14 threads, com *static* próxima em parte das contagens. Ambas distribuem linhas e preservam a localidade do stencil; a diferença observada é compatível com a forma como *guided* reduz os chunks e com a variação residual do ambiente. A versão *static* com *collapse(2)* foi mais lenta:

combinar os laços não trouxe vantagem para essa malha e pode acrescentar custo de indexação, não isolado pela medição.

As estratégias `dynamic` e `guided` preservam o processamento por linhas. `dynamic` acrescenta ordenação para redistribuir uma carga uniforme; `guided` obteve a melhor mediana global, embora a vantagem não seja uniforme em todas as contagens.

A estratégia `dynamic` com `collapse(2)` foi a mais lenta. Com `collapse(2)`, cada chunk de `dynamic`, 16 contém apenas 16 células, enquanto `dynamic` sem `collapse` contém até 16 linhas. Essa redução aumenta as solicitações ao escalonador e é compatível com o overhead observado. A estratégia `guided` com `collapse(2)` foi mais rápida que a versão `dynamic` com `collapse` porque `guided` começa com chunks maiores, reduzindo a frequência dessas solicitações. Os tempos, porém, não isolam o custo de cada mecanismo.

Análise do tamanho do chunk. Para isolar o efeito do tamanho do chunk, a execução foi repetida com oito threads e os volumes equivalentes $\{1, 2, 4, 8, 16, 32, 64, 128\}$ em linhas da grade. Nos laços sem `collapse`, o valor é contado em linhas; nos laços com `collapse(2)`, o chunk contém o número equivalente de células, isto é, o valor da abscissa multiplicado por 510. Cada ponto representa a mediana de cinco execuções.

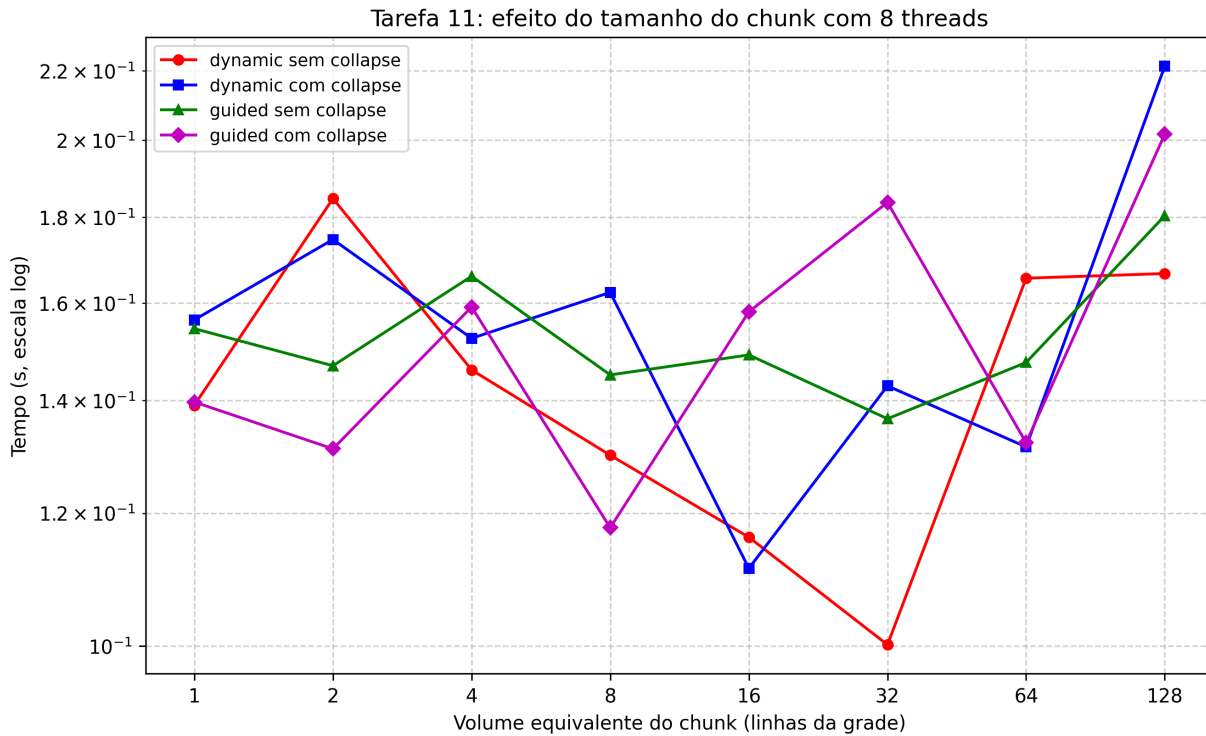


Figure 5: Efeito do tamanho do chunk com oito threads; a abscissa mostra o volume equivalente em linhas.

Efeito do tamanho do chunk. Na Figura 5, aumentar os chunks menores tornou `dynamic` com `collapse` mais rápido, comportamento compatível com a redução do custo de distribuição. No maior volume testado, todas as versões ficaram mais lentas: chunks grandes deixam menos blo-

cos disponíveis para dividir entre as threads. A escolha exige equilibrar granularidade e coordenação; em `guided`, o valor configurado é um limite inferior, e não um tamanho fixo.

6 Conclusão

Os testes validaram a difusão e a equivalência numérica. A melhor configuração observada foi `guided` por linhas, com 9 threads na varredura até 14 threads. `static` por linhas permaneceu competitiva em várias contagens. A piora de `dynamic` com chunks de poucas células aponta o custo de distribuição como principal limitação dessa configuração, sem isolar sua contribuição dos demais custos.

A Código-fonte

```
1 // gcc -std=c11 -O2 -Wall -Wextra -fopenmp v0_seq.c -lm -o ns_v0
2 #include <math.h>
3 #include <omp.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7
8 static void inicializar_campos(double *campo_atual, double *proximo_campo,
9     int tamanho_grid,
10     int modo) {
11     size_t total = (size_t)tamanho_grid * tamanho_grid;
12     double sigma = tamanho_grid / 10.0;
13     for (int i = 0; i < tamanho_grid; ++i)
14         for (int j = 0; j < tamanho_grid; ++j) {
15             double x = i - (tamanho_grid - 1) / 2.0;
16             double y = j - (tamanho_grid - 1) / 2.0;
17             int borda = i == 0 || j == 0 || i == tamanho_grid - 1 || j ==
18                 tamanho_grid - 1;
19             size_t indice = (size_t)i * tamanho_grid + j;
20             campo_atual[indice] =
21                 modo == 2
22                 ? 1.0
23                 : (modo == 1 || borda ? 0.0
24                     : 0.1 * exp(-(x * x + y * y) /
25                         (2.0 * sigma * sigma)));
26         }
27     memcpy(proximo_campo, campo_atual, total * sizeof(*proximo_campo));
28 }
29
30 static int escrever_campo(const char *nome, const double *campo, int
31     tamanho_grid) {
32     size_t total = (size_t)tamanho_grid * tamanho_grid;
33     FILE *arquivo = fopen(nome, "w");
34     if (!arquivo) {
35         perror(nome);
36         return EXIT_FAILURE;
37     }
38     for (size_t indice = 0; indice < total; ++indice)
39         if (fprintf(arquivo, "%.17g%c", campo[indice], (indice + 1) %
40             tamanho_grid ? ' ' : '\n') <
41             0) {
42                 fclose(arquivo);
43                 return EXIT_FAILURE;
44             }
45     return fclose(arquivo) == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
46 }
```

```

43 int main(int argc, char **argv) {
44     const int tamanho_grid = 512;
45     const int num_passos_tempo = 500;
46
47     if (argc > 3 || (argc > 1 && (argv[1][0] < '0' || argv[1][0] > '2' ||
48         argv[1][1] != '\0')))) {
49         fprintf(stderr, "Uso: %s [modo=0|1|2 [campo.txt]]\n", argv[0]);
50         return EXIT_FAILURE;
51     }
52
53     int mode = argc > 1 ? argv[1][0] - '0' : 0;
54     int threads = 1;
55     size_t count = (size_t)tamanho_grid * tamanho_grid;
56
57     double *u = malloc(count * sizeof(*u));
58     double *v = malloc(count * sizeof(*v));
59
60     if (!u || !v) {
61         fprintf(stderr, "Falha de alocação\n");
62         free(u);
63         free(v);
64         return EXIT_FAILURE;
65     }
66
67     const double nu = 0.1, dt = 0.2, dx = 1.0;
68     const double r = nu * dt / (dx * dx); /* 0.02 <= 1/4 em 2D */
69
70     inicializar_campos(u, v, tamanho_grid, mode);
71
72     double start = omp_get_wtime();
73     for (int t = 0; t < num_passos_tempo; ++t) {
74         for (int i = 1; i < tamanho_grid - 1; ++i) {
75             for (int j = 1; j < tamanho_grid - 1; ++j) {
76                 size_t indice = (size_t)i * tamanho_grid + j;
77                 v[indice] =
78                     u[indice] + r * (u[indice - 1] + u[indice + 1] + u[
79                         indice - tamanho_grid] +
80                         u[indice + tamanho_grid] - 4 * u[
81                             indice]);
82             }
83         }
84         double *temporario = u;
85         u = v;
86         v = temporario;
87     }
88
89     double elapsed = omp_get_wtime() - start;
90
91     double peak = u[0], minimum = u[0], sum = 0.0;

```

```

89     for (size_t c = 0; c < count; ++c) {
90         peak = fmax(peak, u[c]);
91         minimum = fmin(minimum, u[c]);
92         sum += u[c];
93     }
94
95     printf("%s tamanho_grid=%d num_passos_tempo=%d modo=%d threads=%d r
96           =%.3f "
97           "tempo=%.9f s minimo=%.17g pico=%.17g soma=%.17g\n",
98           "v0_seq", tamanho_grid, num_passos_tempo, mode, threads, r,
99           elapsed, minimum, peak, sum);
100
101     int status = EXIT_SUCCESS;
102     if (argc == 3)
103         status = escrever_campo(argv[2], u, tamanho_grid);
104     free(u);
105     free(v);
106     return status;
107 }

```

Listing 2: Código sequencial de referência

```

1 #pragma omp for schedule(static)

```

Listing 3: Divisão estática por linhas.

```

1 #pragma omp for collapse(2) schedule(static)

```

Listing 4: Divisão estática com dois laços combinados.

```

1 #pragma omp for schedule(dynamic, 16)

```

Listing 5: Chunks dinâmicos por linhas.

```

1 #pragma omp for collapse(2) schedule(dynamic, 16)

```

Listing 6: Chunks dinâmicos com dois laços combinados.

```

1 #pragma omp for collapse(2) schedule(guided, 16)

```

Listing 7: Chunks guiados com dois laços combinados.

```

1 #pragma omp for schedule(guided, 16)

```

Listing 8: Chunks guiados por linhas.